

protein_loc_organizer

Data extraction of protein localization from online prediction tools

A suite of lightweight Python tools for extracting and standardizing output from various protein localization and property prediction servers. These tools convert non-tabular, inconsistent web-tool outputs into consistent, analysis-ready tables.

This repository aims at making available protein_loc_organizer, a small suite of tools used to produce data tables for the research article “[**PLACEHOLDER:PAPER TITLE**]” (**PLACEHOLDER: DOI**).

Quick Index

- Overview
- Features
- Repo Structure
- Getting Started
- Notice / Disclaimer
- References
- Acknowledgements
- Authors
- License Notice
- Additional Development Notes

Overview

Many online protein prediction tools often present results in formats that are difficult to parse or batch-analyze.

protein_loc_organizer provides a unified set of extractors that:

- Parse raw text, HTML, or downloaded reports from prediction servers;
- Extract key biological features (*e.g.*, signal peptides, localization scores, transmembrane segments) reported by the prediction software;
- Convert them into standardized plain-text tables (CSV, TSV), both human- and machine-readable;
- Facilitate downstream analyses and reproducibility of data organization.

Features

- Standardized output format;
- Standalone extractors for each prediction server/software;
- Minimal dependencies.

Repo Structure

.

```

├─ bepiped_extract.py      # Parser for BepiPred outputs
├─ cello_extract.py        # Parser for CELLO subcellular localization results
├─ lipop_extract.py        # Parser for LipoP predictions
├─ psortb_extract.py       # Parser for PSORTb localization results
├─ signalp_extract.py      # Parser for SignalP outputs
├─ tmhmm_extract.py        # Parser for TMHMM transmembrane predictions
├─ vaxijen_extract.py      # Parser for VaxiJen antigenicity results
├─ virulentpred_extract.py # Parser for VirulentPred outputs
├─ table_builder.py        # Combines parsed outputs into unified table
├─ LICENSE
└─ README.md

```

Getting Started

Requirements

- Python 3.x
- Python modules:
 - collections
 - datetime
 - itertools
 - operator
 - argparse
 - json
 - sys
 - os
 - re
 - numpy
 - pandas

Installation

Just clone this repo and you are good to go:

```
git clone https://github.com/PCas95/protein_loc_organizer.git
```

```
cd protein_loc_organizer
python bepiped_extract.py -h
```

Except for numpy and pandas, all the dependencies are standard Python modules: most likely you will only need to install those two:

```
pip install numpy pandas
```

Or, if you prefer to manage numpy/pandas in a **Conda** environment:

```
conda create -n protein_loc_organizer python=3.11 numpy pandas -y
conda activate protein_loc_organizer
```

run any extractor inside environment:

```
python signalp_extract.py -h
```

Note: since these tools are based on output that may change in the future, building a fixed package distribution (conda or Docker) was deemed unnecessary and potentially misleading for this project. We may consider adding such feature in the future, if there is community interest.

Usage

Each extractor can be run independently or, from the upcoming version, also combined through the wrapper `protein_organizer.py` (not available yet). `table_builder.py` builds a final, merged table with all prediction and additional inferred classification (based on the analysis needs for the publication "").

Supported Tools

- **BepiPred** – epitope prediction
- **CELLO** – subcellular localization
- **LipoP** – signal peptides/lipoprotein detection
- **PSORTb** – subcellular localization
- **SignalP** – signal peptide prediction
- **TMHMM** – transmembrane helices prediction
- **VaxiJen** – antigenicity prediction
- **VirulentPred** – virulence prediction

Notice / Disclaimer

This project is **open-source** and is **not affiliated with, endorsed by, or sponsored by** the developers or maintainers of any of the prediction tools it parses. All trademarks and tool names belong to their respective owners.

The extractors in this suite are provided solely to help users organize and standardize results for downstream analysis. They operate on outputs generated by third-party prediction servers that **do not provide standardized or tabular output formats**, and that **do not provide public code or stable programmatic interfaces**.

This project acts only as an **independent** utility to structure information from prediction servers for research workflows.

Because these prediction tools may change their underlying logic, version, or webpage layout **without notice**, any such modification may cause the extractors in this repository to become **incompatible** with future outputs.

Therefore, users are responsible for complying with the terms of use of each prediction tool and for verifying the accuracy and continued compatibility of the extracted data.

References

I. Krasteva, [...]; "". <https://doi.org/>.

Acknowledgements

Many thanks to the Proteomics and Serology Unit, in particular to Ivanka Krasteva and Federica d’Onofrio, for collaboration and feedback.

Authors

Pierluigi Castelli

License Notice

Every script in the this repo is open source and free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

The software is distributed for transparency of research methods, and in hope that it will be useful to others, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE (see “Disclaimer” section).

See the GNU Affero General Public License for more details.

Additional Development Notes

The extractors in this repository follow a common design: they take highly unstructured, web-derived prediction results and coerce them into a consistent, machine-readable format. Because each prediction server outputs data differently (and often without a formal specification), the internal logic of each extractor must be adapted to the quirks of that tool’s output.

Example: the BepiPred extractor (`bepipred_extract.py`) performs explicit header validation, reconstructs per-position data into nested dictionaries, and applies domain-specific filtering rules (*e.g.*, probability thresholds, solvent exposure, and minimum run lengths).

Similar patterns are used throughout the suite, but each extractor necessarily implements tool-specific parsing rules depending on what information the original source provides.

Structure and Expectations

- **Input is inherently unstructured**

Most supported tools provide text as webpages, *ad-hoc* CSVs, or inconsistent HTML layouts. Because of this, some parts of the code rely on fragile assumptions (column order, labels, spacing, or hidden formatting choices) that cannot be made more robust without official schemas.

- **Extractors are intentionally lightweight**

Each extractor is an independent script. This keeps them simple to inspect, debug, and update when upstream tools change their format.

- **Error handling is defensive**

Since upstream tools may silently alter their output, extractors explicitly validate headers, field counts, or expected keywords early. This prevents silently producing corrupted tables. **Always check the output!**

- **Output is standardized, input never is**

All extractors aim to emit predictable, tidy, CSV/TSV outputs. Ensuring this consistency often means compensating for missing values, renaming fields, or flattening nested patterns.

Known Limitations

Some parts of the parsing logic could theoretically be more elegant, generalizable, or modular, but cannot be improved meaningfully because the input formats themselves have no guarantees. As long as upstream tools continue to provide:

- undocumented formats
- *ad-hoc* HTML layouts
- inconsistent column structures
- hard-coded symbols or abbreviations

There are sections of each extractor that look brittle or overly manual. This is intentional: building more robust functions is not a reliable way to ensure correct extraction from shifting, unstructured data sources, in this specific case (see Future Improvements below).

Future Improvements

Coming up in the next version(s):

- Change line extraction from brittle, regex-based logic to list-based > Users will provide a list of SeqIDs; tools will extract them using a smart and consistent logic .
- Unify the overall architecture > Scripts are currently undergoing massive reorganization of core logic, to provide a unified, structured, shared logic, allowing easier management through a wrapper.
- Factor out common parsing utilities by adding a small `common_utils.py` shared module with helpers
- Normalize error handling and messages
- Standardize output table format (CSV/TSV) > Currently the separator used is chosen case-by-case, depending on the kind of characters in the columns (noisy or may contain commas) and the needs of the original wet-lab team.
- Improve type hints and docstrings consistently
- Add tiny example inputs

Contributing

If you wish to contribute extractors for additional prediction tools, you can:

1. Open an issue describing the tool you want to support, including example outputs if possible.
2. Submit a pull request with a new `*_extract.py` script following the structure and conventions used in the existing extractors.
3. Add documentation explaining the new tool and its expected output fields.

Contributors adding new extractors are encouraged to follow the general pattern:

- Validate the expected header or a minimal set of identifying fields.
- Build an internal data structure representing each protein and its per-position/per-feature metadata.
- Apply tool-specific filtering or interpretation rules.
- Produce clean CSV output compatible with `table_builder.py`.

Given the instability of upstream formats, **simplicity and clarity take precedence over abstraction**, at this time.

Contributions of bug fixes, improvements, and test cases are also welcome. Please feel free to discuss ideas or ask questions via the issue tracker before starting major work.

Final Note

The main purpose of this repository is to provide access to the code used to produce data table in the manuscript “[**PLACEHOLDER:PAPER TITLE**]” (**PLACEHOLDER: DOI**).

The scripts are release for transparency and to help the community, but the most desirable outcome would be the release of a **tested, open-source, published, reproducible and documented bioinformatics pipeline for proteomics**.